Appendix A1 · Actuation and the Control Effectiveness Matrix

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-04

ℹ Recordings of this semester's own classes

Every session of this course is recorded as it is given and posted to the playlist below. The written material here is the reference; the recording is the lecture, at the pace it was actually delivered. Use it to review a derivation, or to catch up on a missed week before the laboratory.

	Where
[충남대] 2026 센서신호처리 — this course, recorded live, 2026	playlist

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

✎ Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	`winget install --id Git.Git -e`
once	the folder that will hold the course, e.g. `Documents`	`git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git`
before every class	inside the cloned folder `Sensor-Signal-Processing-and-Fusion`	`git pull`

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder, placed so that `Tools\MSS\VESSELS\otter.m` exists: `_tools\mss_path.m` searches for that one file and puts the toolbox on the MATLAB path once it finds it. The current MSS release keeps `otter.m` under `CRAFT\USV\models\` instead, where the course scripts do not find it.
- **A 118 MB folder disappears on the first pull after 2026-10-27.** `lectures\MSS-master` held a copy of MSS until then. That copy used the newer layout above, so no course script could reach it, and it only made cloning slower. Its removal is expected and is not a fault. Any file kept inside that folder must be moved elsewhere before pulling.

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This appendix**: ① one rule that produces $\mathbf{B}$ for any thruster layout ② the attainable control set and what actuation rank costs ③ the command that looks like a pure turn and is not, and the one that is

★ When to read this

- This appendix is **optional for Weeks 3 and 4**. Those weeks need only two results from it, and both are quoted where they are used: the vessel can produce $X \in [-133.42,\ 239.36]$ N, and its yaw moment is $N = y_{\text{pont}}(T_1 - T_2)$.
- It becomes **required before Week 7**, where the allocation problem stops having an obvious answer, and before Weeks 11 to 12, where the thruster layout changes three times.
- Prerequisites: Week 1 — the twelve states, $T = k\,n|n|$ with $k_{\text{pos}} \neq k_{\text{neg}}$, and the observation that $Y \equiv 0$. Here that observation becomes a theorem.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Construct the control effectiveness matrix $\mathbf{B}$ of an arbitrary planar thruster layout from the position and direction of each thruster, without consulting a reference.

2. State the rank of $\mathbf{B}$ for a given layout and name the degree of freedom that a rank deficiency removes.
3. Explain why a tilting thruster contributes two columns rather than one, and why that choice makes $\boldsymbol{\tau} = \mathbf{B}\mathbf{f}$ linear.
4. Compute and plot the attainable control set of a layout, and read from it the limits a controller will later have to respect.
5. Derive the propeller-speed pair that produces a yaw moment with zero net surge force, and verify it against the plant.
6. Distinguish a zero **row** of $\mathbf{B}$ from a zero **column** of $\mathbf{M}^{-1}$, and give the measured consequence of each.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink
Toolboxes	none beyond Simulink for this week
MSS	`Tools/MSS`, added by the setup script
Course folder	`GradCourse/lectures/A1_simulink`
Expected duration	60 min theory, 60 min laboratory

Part 1 · Theory

Where this appendix goes

Seven sections, all of them consequences of **one rule** stated in A1-2. Read that rule carefully and the rest follows.

	Question	Sections
1	**Why does anything sit between a controller and a propeller?**	A1-1
2	**The column rule.** One thruster at (x, y) pushing along $\mathbf{e}$ contributes exactly one column of $\mathbf{B}$	A1-2
3	**What the rule settles before any arithmetic.** The rank says which demands are reachable; a tilting thruster contributes two columns, not one	A1-3, A1-4
4	**Three results that surprise people.** A symmetric command is not a symmetric force; the reachable set is a polygon, not a box; an empty row of $\mathbf{B}$ is not an empty column of its inverse	A1-5 to A1-7

$$\mathbf{b}_i = \begin{bmatrix} e_x & e_y & x_i e_y - y_i e_x \end{bmatrix}^\top$$

Every $\mathbf{B}$ in this course is produced by that expression — none is transcribed from another file. The Otter's sway row is empty because both propellers face forward and $e_y = 0$, not because someone decided it should be.

This appendix is a prerequisite for Week 7 and is referenced from Weeks 3, 4 and 5 wherever a demanded force has to become a shaft speed.

A1-1. The problem this week solves

- Week 1 wrote the generalised force of the base Otter as a 3×2 matrix and moved on. That matrix was asserted. It has to be derived, for three reasons.

Reason	Where it is felt
Weeks 11 to 12 change the thruster layout three times	a transcribed matrix would have to be re-derived and re-checked three times
This project already ships two mutually **negative** $\mathbf{B}$ matrices	a transcribed matrix cannot be told apart from its own sign error
Week 7 inverts $\mathbf{B}$	an inverse is only as trustworthy as the matrix it inverts

> **✕ Two contradictory matrices exist in this repository**
>
> `Lecture/_tools/otter4_B.m` and `otter_params.m` line 41 differ by an overall sign. Neither is imported into this course. Every $\mathbf{B}$ used from this week onward is produced by `_tools/otter_B.m` from the single rule of §A1-2, so a sign convention cannot enter twice.

A1-2. The column rule

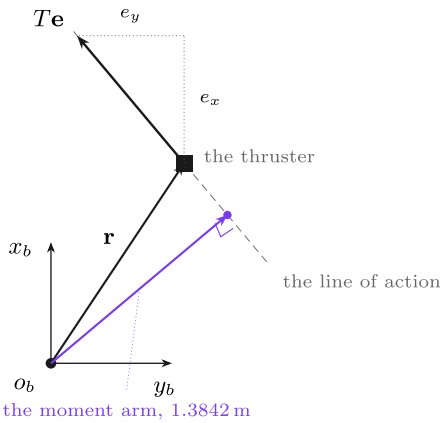
- Consider one thruster fixed to the hull at the point $\mathbf{r} = (x, y)$ in $\{b\}$, pushing along the unit vector $\mathbf{e} = (e_x, e_y)$, with scalar thrust T.
- Its contribution to the planar generalised force is the force itself and the moment of that force about the origin of $\{b\}$:

$$\begin{bmatrix} X \\ Y \end{bmatrix} = T \begin{bmatrix} e_x \\ e_y \end{bmatrix},$$
$$N = (\mathbf{r} \times T\mathbf{e})_z = T\,(x\,e_y - y\,e_x).$$

- Collecting the three:

$$\boxed{\boldsymbol{\tau}_i = T_i \begin{bmatrix} e_x \\ e_y \\ x\,e_y - y\,e_x \end{bmatrix}}$$

- The bracketed vector is the generalised force produced by **one newton** from that thruster. It is one column of $\mathbf{B}$, and it depends only on geometry.



the rule

one newton from a thruster at (x, y) pointing along $\mathbf{e}$:

$$\mathbf{b} = [\,e_x, \quad e_y, \quad x\,e_y - y\,e_x\,]^\top$$

and that column depends on geometry alone.

n thrusters, n columns: $\boldsymbol{\tau} = [\,X \quad Y \quad N\,]^\top = \mathbf{B}\mathbf{f}$

The check that makes the rule believable. With $x = 1.2\,\text{m}$, $y = 0.8\,\text{m}$ and $\mathbf{e}$ $40°$ to port of the bow, $xe_y - ye_x = \mathbf{-1.3842}$, and the violet arm measures $\mathbf{1.3842}\,\text{m}$. The third entry is not a new idea — it is the ordinary *force times perpendicular distance*, written so the sign comes out on its own.

The base Otter, from the same rule. Both propellers face forward, so $\mathbf{e} = (1, 0)$, and they sit at $x = 0$, $y = \mp 0.395\,\text{m}$, giving $xe_y - ye_x = \pm 0.395$. Here $e_y = 0$ for both, so the middle row of $\mathbf{B}$ is empty — and the Otter can never be pushed sideways.

e must be a unit vector. Entering $(2, 0)$ instead of $(1, 0)$ silently doubles that thruster's authority, and every rank test still passes.

Reading the figure

Element	Meaning
grey axes	the body frame, origin o_b, with x_b towards the bow and y_b to starboard
orange arrow	the position vector $\mathbf{r} = (x, y)$ from o_b to the thruster
black square	the thruster
blue arrow	its thrust $T\mathbf{e}$, and the two dashed segments are the components e_x and e_y
grey dashed line	the **line of action** — the thrust extended in both directions
violet arrow	the perpendicular from o_b to that line, with the right-angle mark

What the figure says

The first two entries of the column are just the thrust resolved into the body axes and need no picture. The third entry is the one worth drawing.

That entry is $xe_y - ye_x$, and the claim is that it *is* the moment arm — the perpendicular distance from the origin to the thruster's line of action. The figure checks that claim rather than asserting it. With $x = 1.2$ m, $y = 0.8$ m and $\mathbf{e}$ pointing $40°$ to port of the bow, the formula gives -1.3842. The violet perpendicular, measured on the same drawing, is 1.3842 m. **They agree exactly.**

The reason is that $xe_y - ye_x$ is the z component of $\mathbf{r} \times \mathbf{e}$, and the magnitude of a cross product with a *unit* vector is precisely the perpendicular distance to the line. So the third entry is nothing more exotic than "force times

perpendicular distance" from school — written in a form that produces the **sign** automatically instead of leaving it to be argued about. Arguing about that sign is where hand derivations of $\mathbf{B}$ usually go wrong.

Notice what does *not* appear anywhere in this. Only $\mathbf{r}$ and $\mathbf{e}$ change from thruster to thruster: no hydrodynamics, no calibration, no measured data. That is why $\mathbf{B}$ can be written down before the vessel exists.

Setting $\mathbf{e} = (1, 0)$ and $x = 0$ collapses the arm to $-y$, and the Otter's two propellers at $y = \mp 0.395$ m give ± 0.395. The middle entry e_y is zero for both — and **that empty row is the whole of §A1-3.**

- For n thrusters the contributions add, because forces add:

$$\boldsymbol{\tau} = \begin{bmatrix} X \\ Y \\ N \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_n \end{bmatrix}}_{\mathbf{B}} \underbrace{\begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_n \end{bmatrix}}_{\mathbf{f}} = \mathbf{B}\,\mathbf{f}$$

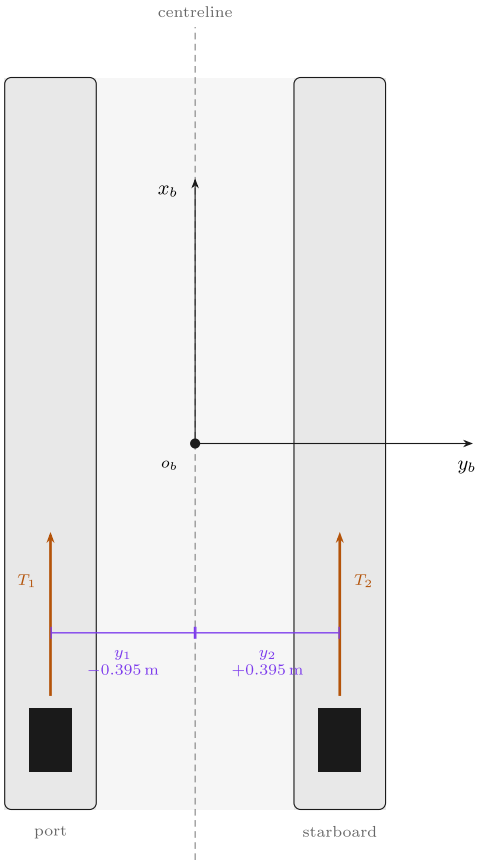
Symbol	Quantity	Value / source
$\mathbf{r} = (x, y)$	thruster position in $\{b\}$	`otter_config`, field `pos`
$\mathbf{e}$	unit thrust direction in $\{b\}$	`otter_config`, field `dir`
T_i	scalar thrust of thruster i	$k\,n_i\lvert n_i\rvert$, Week 1
$\mathbf{B}$	control effectiveness matrix	`_tools/otter_B.m`

> **⚠ e must be a unit vector**
>
> The column rule assumes $\lVert\mathbf{e}\rVert = 1$. A direction entered as $(2, 0)$ instead of $(1, 0)$ silently doubles that thruster's authority and leaves every rank test passing. `otter_B.m` normalises before applying the rule; a hand derivation has no such protection.

The base Otter

The Otter from above — where the propellers are, and why B looks the way it does

A catamaran: two pontoons, one propeller in each, both bolted facing forward. Every entry of **B** follows from this picture and nothing else.



reading B off the picture

One propeller at (x_i, y_i) pushing along the unit vector $\mathbf{e}_i = (e_x, e_y)$ contributes exactly one column (`_tools/otter_B.m`, the column rule):

$$\mathbf{b}_i = \begin{bmatrix} e_x \\ e_y \\ x_i e_y - y_i e_x \end{bmatrix}$$

Both propellers are bolted facing forward, so $\mathbf{e}_i = (1, 0)$ for both and the picture supplies only $y_1 = -0.395$ and $y_2 = +0.395\,\text{m}$.

Surge. Both push forward: $e_x = 1$ twice, so the first row is $[1 \;\; 1]$.

Sway. Neither has any component across the hull, $e_y = 0$, so the second row is empty: $Y = 0$ — **exactly, always**. No shaft speed can produce sway.

Yaw. The third entry is $-y_i$, which is the **negative** of the coordinate marked on the drawing: the port propeller sits at $y = -0.395$ and turns the bow to *starboard*, which is positive yaw.

The along-ship station x_i never appears, because it is multiplied by $e_y = 0$. The propellers may be drawn anywhere along their pontoons and **B** is unchanged.

$$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ 0.395 & -0.395 \end{bmatrix}, \qquad \boldsymbol{\tau} = \mathbf{B}\,\mathbf{f}$$

The hull is drawn here at $\psi = 0$, so $\{b\}$ and $\{n\}$ happen to line up. At any other heading the two triads differ by $\mathbf{R}(\psi)$ and **B does not change at all** — it is written entirely in $\{b\}$. That is the whole reason the allocation can be designed once and used at every heading.

Reading the figure

Element	Meaning
x_b, y_b	the body axes, origin o_b at midship on the centreline
blue rectangles	the two propellers, one per pontoon
orange arrows	their thrust, both along $+x_b$
∓ 0.395 m	the moment arms, equal and opposite about the centreline

- Both propellers face forward, so $\mathbf{e} = (1, 0)$ for each. They sit on the pontoons at $x = 0$, $y = \mp y_{\text{pont}}$ with $y_{\text{pont}} = 0.395$ m. The rule gives

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \cdot 0 - (-0.395) \cdot 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ +0.395 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 1 \\ 0 \\ -0.395 \end{bmatrix},$$

$$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ 0.395 & -0.395 \end{bmatrix}, \qquad \operatorname{rank}(\mathbf{B}) = 2.$$

- This is the matrix Week 1 asserted, now obtained from geometry alone.

A1-3. What the rank means

- $\mathbf{B} \in \mathbb{R}^{3\times n}$ maps thrust to generalised force. Its **column space** is exactly the set of $\boldsymbol{\tau}$ that any combination of thrusts can produce.

$\operatorname{rank}(\mathbf{B})$	Name	Consequence
3	fully actuated in the plane	every $\boldsymbol{\tau} \in \mathbb{R}^3$ is producible; direction and magnitude are independent
2	underactuated	one direction of $\mathbb{R}^3$ is unreachable; guidance must produce that motion indirectly
< 2	degenerate	the vessel cannot be steered by these actuators alone

- For the base Otter the unreachable direction is $\mathbf{Y}$. The middle row of $\mathbf{B}$ is identically zero, so

$$Y = \mathbf{0}^{\top}\mathbf{f} = 0 \qquad \text{for every } \mathbf{f} \in \mathbb{R}^2.$$

> **★ A zero row is a theorem, not a measurement**
>
> $Y = 0$ does not depend on the thrust, on the propeller curve, on saturation, or on any gain. It follows from the fact that both propellers point along $\mathbf{e} = (1, 0)$, so neither has any component across the hull. No controller can be tuned to produce a sideways force, because there is nothing to tune.

- The pseudo-inverse does not rescue anything. Asking for one newton of pure sway gives

$$\mathbf{B}^{\dagger}\begin{bmatrix}0\\1\\0\end{bmatrix} = \begin{bmatrix}0\\0\end{bmatrix},$$

- that is, the least-squares allocator correctly answers *do nothing*, because doing nothing is the closest reachable point to a request that lies outside the column space. Week 7 makes this precise.

A1-4. Tilting thrusters and the extended thrust vector

- A thruster that can rotate has two unknowns, its thrust T_i and its azimuth δ_i, and it enters the moment through $\sin\delta_i$ and $\cos\delta_i$. Written that way the allocation problem is **not** linear in the unknowns.
- Writing the same thrust in Cartesian components removes the trigonometry:

$$\mathbf{f}_i = \begin{bmatrix}f_{ix}\\f_{iy}\end{bmatrix} = \begin{bmatrix}T_i\cos\delta_i\\T_i\sin\delta_i\end{bmatrix}, \qquad T_i = \sqrt{f_{ix}^2 + f_{iy}^2}, \qquad \delta_i = \operatorname{atan2}(f_{iy},\, f_{ix}).$$

- The thruster then contributes **two** columns, the $\mathbf{e} = (1, 0)$ column and the $\mathbf{e} = (0, 1)$ column of the same location:

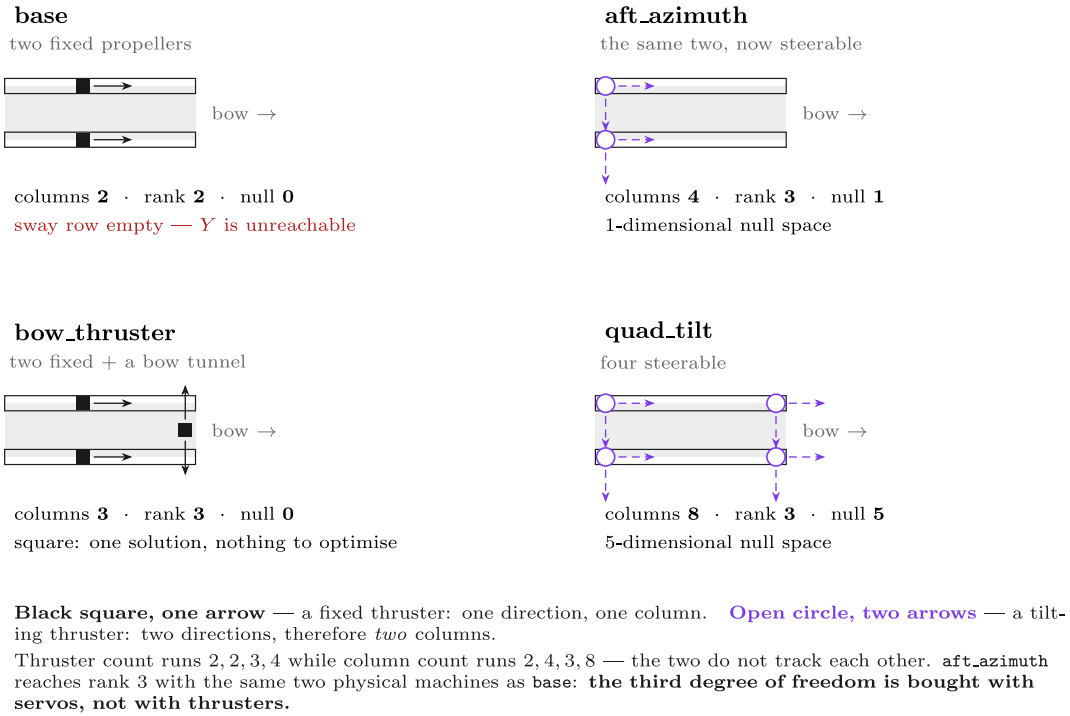
$$\boldsymbol{\tau}_i = \begin{bmatrix}1 & 0\\0 & 1\\-y & x\end{bmatrix}\begin{bmatrix}f_{ix}\\f_{iy}\end{bmatrix}.$$

- This is the **extended thrust vector** of Fossen (2011) §12.3.4. The azimuth is recovered after the allocation rather than solved for during it.
- The cost is that the two components of one machine are not independent: $\sqrt{f_{ix}^2 + f_{iy}^2} \le T_{\max}$ is a circular constraint, and δ_i is subject to a mechanical limit and a rate limit. Week 7 handles both; this week only records that the linear structure has been bought and what it was bought with.

The four layouts of this course

- All four share the same hull. Only the actuator model changes, so any later comparison measures the actuation and nothing else.

Configuration	Thrusters	Columns of $\mathbf{B}$	Rank	Null-space dimension	Week
`base`	2 fixed	2	2	0	1–8
`aft_azimuth`	2 tilting	4	3	1	9
`bow_thruster`	3 fixed	3	3	0	10
`quad_tilt`	4 tilting	8	3	5	11



Black square, one arrow — a fixed thruster: one direction, one column. **Open circle, two arrows** — a tilting thruster: two directions, therefore *two* columns.
Thruster count runs $2, 2, 3, 4$ while column count runs $2, 4, 3, 8$ — the two do not track each other. `aft_azimuth` reaches rank 3 with the same two physical machines as `base`: **the third degree of freedom is bought with servos, not with thrusters.**

Reading the figure

Element	Meaning
grey outline	the hull, drawn identically in all four; the two pontoons are at $y = \mp 0.395$ m
black square with one blue arrow	a **fixed** thruster — one direction, therefore one column
black square with two blue arrows	the **tunnel** thruster, fixed across the hull, $\mathbf{e} = (0, 1)$
violet circle with two arrows	a **tilting** thruster — two directions, therefore two columns
the numbers under each hull	columns, rank and null-space dimension, as printed by section C

What the figure says

Four actuator layouts on one hull, drawn to the same scale from the coordinates in `otter_config.m`. Each panel answers one question: how many independent things can this arrangement ask the vessel to do?

The counts are worth reading carefully, because two of them do not track each other. Thruster count goes $2, 2, 3, 4$ while column count goes $2, 4, 3, 8$ — a **tilting** machine contributes two columns and a fixed one contributes one. Rank goes $2, 3, 3, 3$, so every layout except the first can produce all three planar forces.

The first panel shows why it is the exception. Both `base` arrows point the same way, along $\boldsymbol{x}_b$. No sum of two parallel vectors has a component across the hull, so sway is unreachable **for reasons of geometry, not of size.** A bigger propeller does not help. A differently pointed one does.

That is exactly what the second panel is: the same two machines, moved aft and made steerable. Nothing was added, and the rank goes from 2 to 3. **Actuation authority is a property of directions, not of thrust** — the strongest claim in this appendix, and one panel apart from the one that contradicts it.

The last two panels separate for a different reason. `bow_thruster` is square, three columns for three demands, so the allocation has exactly one answer and there is nothing to choose. `quad_tilt` has eight columns for three demands, so its answers form a five-dimensional set and **something has to decide between them**. That decision is what Week 7 is for.

- Two observations follow immediately, and both matter later.
  - `aft_azimuth` reaches rank 3 with **the same two physical machines** as `base`. The third degree of freedom is bought with servos, not with thrusters.
  - `bow_thruster` has a **square** $\mathbf{B}$ with rank 3, so the allocation has a unique solution and there is nothing to optimise. `quad_tilt` has a five-dimensional null space, so it has infinitely many solutions and Week 7's machinery finally has something to do.

A1-5. Symmetry in the command is not symmetry in the force

- The command $n = [n_1, -n_1]$ looks like a pure turn. It is not, because the propeller curve is not odd:

$$X = k_{\text{pos}}\, n_1^2 + k_{\text{neg}}\,(-n_1)|-n_1| = (k_{\text{pos}} - k_{\text{neg}})n_1^2 \;\neq\; 0.$$

- With $n_1 = 60$ rad/s this leaves $X = 16.686$ N, and the vessel creeps forward while it turns.
- Requiring $X = 0$ instead of assuming it gives the pair that does produce a pure turn:

$$k_{\text{pos}}\, n_1^2 = -k_{\text{neg}}\, n_2|n_2| \quad\Longrightarrow\quad n_2 = -n_1\sqrt{\frac{k_{\text{pos}}}{k_{\text{neg}}}} = -60\sqrt{1.7192} = -78.6701\ \text{rad/s}.$$

Symbol	Quantity	Value / source
k_{pos}	forward bollard coefficient, one propeller	0.011080, `otter.m`
k_{neg}	reverse bollard coefficient	0.006445, `otter.m`
$\sqrt{k_{\text{pos}}/k_{\text{neg}}}$	correction factor on the reverse shaft	1.3112
n_2	corrected reverse shaft speed for $n_1 = 60$	-78.6701 rad/s

- The corrected command is not merely cleaner. It also produces **more** yaw moment, because both propellers now push with the full 39.888 N instead of the reverse propeller contributing only -23.202 N:

$$N = y_{\text{pont}}\,(T_1 - T_2) = 0.395\,(39.888 - (-39.888)) = 31.512\ \text{N·m},$$

- against 24.921 N·m for the naive command, an increase of 26.4%.

> ✎ **The correction is free**
>
> Removing the unwanted surge force and increasing the yaw moment are the same act. Nothing is traded away except reverse shaft speed, and -78.67 rad/s is well inside the limit $n_{\min} = -101.74$ rad/s. A layout in which the correction did not fit inside the saturation box would be a layout with a genuine trade-off; this one is not.

A1-6. The attainable control set

- The reachable set of generalised forces is the image of the shaft-speed box under the propeller curve followed by $\mathbf{B}$:

$$\mathcal{T} = \{\, \mathbf{B}\,\mathbf{f}(\mathbf{n}) \;:\; n_i \in [n_{\min},\, n_{\max}] \,\} \subset \mathbb{R}^3.$$

- Three properties of $\mathcal{T}$ are worth stating before any controller is designed.

Property	For the base Otter
dimension	2, not 3 — it is a flat region lying in the plane $Y = 0$
symmetry	none. $X \in [-133.42,\ 239.36]$ N, a ratio of 1.794
coupling	X and N are not independent. Demanding the largest $\lvert N \rvert$ fixes X

- The fore-aft ratio is **not** $k_{\text{pos}}/k_{\text{neg}} = 1.719$. The saturation limits are asymmetric as well, $n_{\max} = 103.93$ against $n_{\min} = -101.74$, and the two effects multiply:

$$\frac{X_{\max}}{\lvert X_{\min} \rvert} = \frac{k_{\text{pos}}\, n_{\max}^2}{k_{\text{neg}}\, n_{\min}^2} = 1.7192 \times \left(\frac{103.93}{101.74}\right)^2 = 1.794.$$

> **Why the set matters**
>
> Every controller designed from Week 3 onward produces a demanded $\boldsymbol{\tau}$. If that demand lies outside $\mathcal{T}$, the actuators saturate and the loop no longer behaves as designed — this is the mechanism behind the integrator windup of Week 3. Knowing the shape of $\mathcal{T}$ in advance is what makes a saturation event diagnosable rather than mysterious.

A1-7. An empty row in B is not an empty column in $\mathbf{M}^{-1}$

- Section A1-3 proved $Y \equiv 0$. It is tempting to conclude that the vessel cannot move sideways. That conclusion is wrong, and the reason is instructive.
- At rest the Coriolis and damping terms both vanish, so the plant reduces to a linear map:

$$\dot{\boldsymbol{\nu}} = \mathbf{M}^{-1}\boldsymbol{\tau}, \qquad \boldsymbol{\nu} = \mathbf{0}.$$

- $\mathbf{M}$ is **not diagonal**. The payload places the centre of gravity at $x_g = 0.153$ m rather than at the origin of $\{b\}$, and transferring the rigid-body inertia from the CG to the origin fills in the sway-yaw entry:

$$M_{26} = M_{62} = (m + m_p)\, x_g = 80 \times 0.153125 = 12.25\ \text{kg·m}.$$

- A pure yaw moment therefore produces a sway **acceleration**, with no sway force anywhere in the problem:

$$\dot{v} = \left(\mathbf{M}^{-1}\right)_{26} N = -1.3046 \times 10^{-3}\ \frac{\text{m/s}^2}{\text{N·m}}.$$

Statement	Mechanism	Regime
$Y \equiv 0$	geometry — the row of $\mathbf{B}$ is empty	always
$\dot{v} \neq 0$ under pure N	mass coupling M_{26}	the first instants, while $\dot{\boldsymbol{\nu}} \neq 0$
$v \neq 0$ in a steady turn	Coriolis balanced by cross-flow drag	the steady state, where $\mathbf{M}$ leaves the balance entirely

- Week 1 attributed the sway velocity of a turn to the Coriolis term. For the **steady** value that is correct: setting $\dot{\boldsymbol{\nu}} = \mathbf{0}$ removes $\mathbf{M}$ from the equation. This section is about the transient, and both mechanisms are real.

> **★ Actuation rank and controllability are different questions**
>
> A rank-2 $\mathbf{B}$ says the vessel cannot be *pushed* sideways. It does not say the vessel cannot *reach* a point to its side — it plainly can, by turning and driving. What underactuation costs is the ability to do so **without changing heading**, and that is exactly what Week 9 needs for dynamic positioning and cannot have from this hull.

Part 2 · Laboratory

A. Setting up and running (10 min)

```
cd GradCourse/lectures/A1_simulink
A1_0_setup
```

Expected output:

```
  A1 setup complete
    configuration   base, 2 thrusters, 2 columns, rank(B) = 2
    B  = [ 1.000  1.000 ;  0.000  0.000 ;  0.395 -0.395]
    sway row        max |B(2,:)| = 0.0e+00   -> Y is unreachable
    n_cmd           [60 ; -60] rad/s
    simulation      60 s at h = 0.02 s
```

- `A1_0_setup.m` is **the only file to edit in this appendix**. Note that `B_alloc` is not typed in; it is `cfg.B`, produced by `otter_B` from the column rule.
- To restore a model that has been broken: `A1_1_build_actuation`.

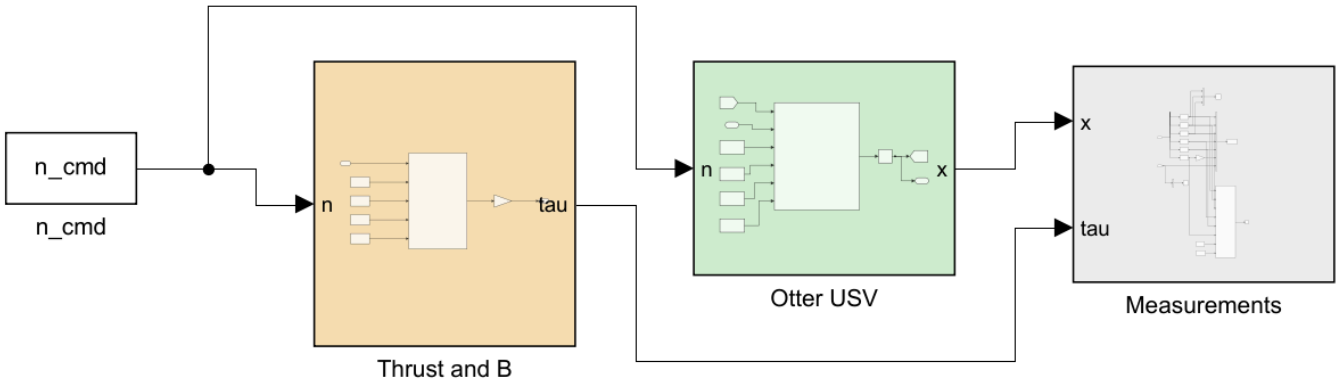
The files of this appendix, in the order the sections use them

Order	File	Section	What it produces
0	`A1_0_setup.m`	A	the base workspace, so the model can be run from Simulink
1	`A1_1_build_actuation.m`	B	`A1_actuation.slx` and `img/A1_actuation.png`
C	`A1_C_four_layouts.m`	C	the rank table and the pseudo-inverse check — no simulation
D	`A1_D_attainable_set.m`	D	`img/A1_result_set.png`
E	`A1_E_command_that_turns.m`	E	`img/A1_result_turn.png`
F	`A1_F_sway_without_force.m`	F	`img/A1_result_coupling.png`

- Each section is one script. Running a section leaves exactly the numbers and the one figure that section discusses, so a class can work through the appendix a page at a time.
- Sections C to F call `A1_1_build_actuation` themselves if the model is missing, so any one of them can be run first.
- The remaining files — `A1_vars.m`, `A1_read.m`, `A1_plot.m`, `A1_animate.m` — are called **by** the scripts above and by the model. They are never run by hand.

B. Reading the model (15 min)

> **To produce this figure**
>
> `A1_1_build_actuation` writes `A1_simulink/img/A1_actuation.png` at the end of the build. The diagram belongs to the builder and to nothing else, so it changes when the model changes and not when a gain changes.




```
APPENDIX A1  -  ACTUATION AND THE CONTROL EFFECTIVENESS MATRIX

Two paths leave the same command. The upper path is this appendix's
arithmetic: shaft speed -> thrust -> generalised force, tau = B f.
The lower path is the unmodified plant. They must agree.

THE COLUMN RULE

A thruster at (x, y) pushing along the unit vector e contributes
exactly one column of B:

      [ e_x ; e_y ; x e_y - y e_x ]

For the base Otter both propellers face forward at y = -+0.395,
so B = [1 1 ; 0 0 ; 0.395 -0.395].  Rank 2, and the middle row
is EMPTY. No propeller speed produces a sideways force.

THREE THINGS TO TRY

1  n_cmd = [60; 60].   Y stays at zero to machine precision.

2  n_cmd = [60; -60].  The command that looks like a pure turn.
   X is 16.686 N, not zero, because k_pos is not k_neg.

3  n_cmd = [60; -78.6701].  The command that IS a pure turn.
   X is zero AND N is 26 per cent LARGER than in case 2.

Rank 2 is the whole story of this course. Weeks 11 to 12 buy the
third rank back, one hull at a time, and measure what it costs.
```

Reading the figure

Element	Meaning
`n_cmd` (white)	the constant propeller command $[n_1; n_2]$ in rad/s
`Thrust and B` (sand)	inside: $T = kn\|n\|$ with saturation applied to n, then $\boldsymbol{\tau} = \mathbf{B}\mathbf{f}$
`Otter USV` (green)	`otter.m`, called unchanged, fed the **same** command
`Measurements` (grey)	selectors, scope, workspace log and live view

- The diagram has **two paths from one command**. The upper path is this appendix's arithmetic; the lower path is the unmodified plant. The colour code separates them: sand is actuation and allocation, green is the motion model.

- The two paths are never wired together. They are compared by their results, which is the only comparison that can fail.
- Opening `Thrust and B` shows two blocks: the thruster model, and a Gain block in matrix mode whose value is `B_alloc`, produced by `otter_B` from the column rule of §A1-2. Nothing is transcribed.

> **⚠ Saturate the shaft speed, not the thrust**
>
> The thruster block clips n before applying the thrust curve. Clipping T instead would place the reverse limit at a force the propeller can never produce, because k_{pos} and k_{neg} differ. The order of the two operations is not interchangeable.

C. The four layouts (15 min)

```
A1_C_four_layouts
```

The table is produced from `otter_config` alone; no simulation is involved.

Configuration	Thrusters	Columns	Rank	Null	$\max\|B(2,:)\|$
`base`	2	2	2	0	**0.000**
`aft_azimuth`	2	4	3	1	**1.000**
`bow_thruster`	3	3	3	0	**1.000**
`quad_tilt`	4	8	3	5	**1.000**

- Only `base` has an empty sway row, and only `base` has rank 2. These are the same statement.
- The pseudo-inverse check, printed by the same script:

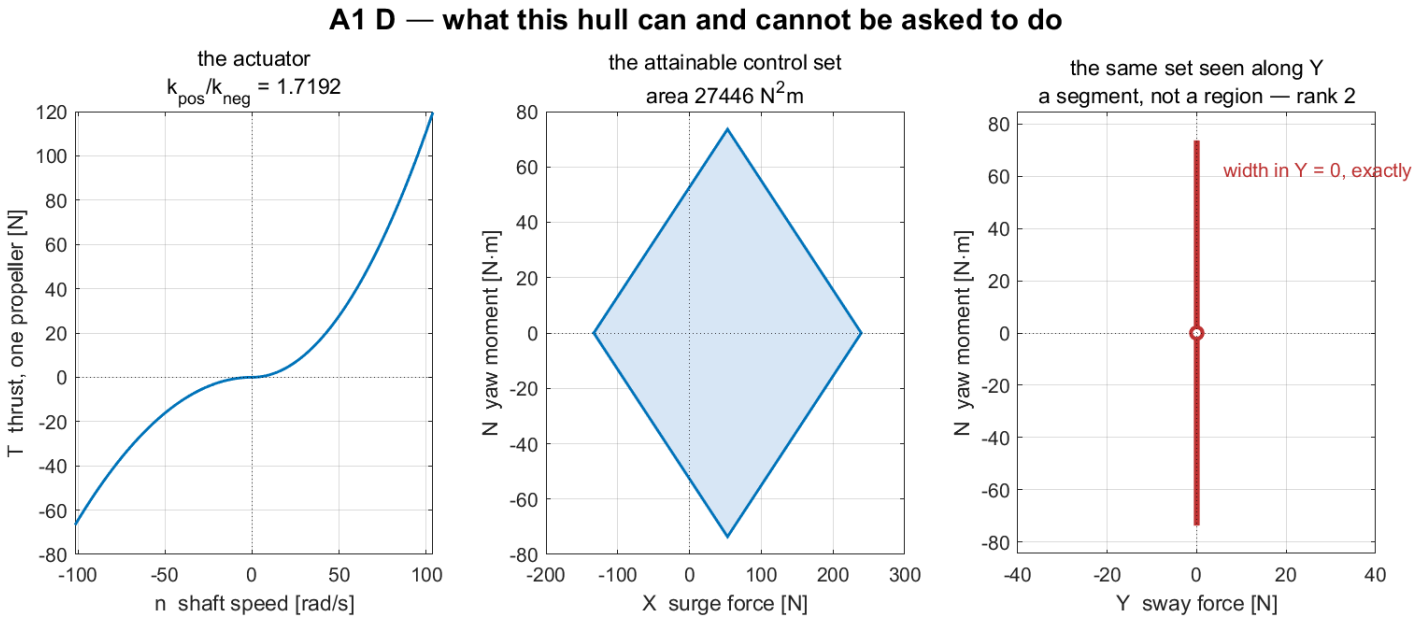
```
pinv(B) * [0 1 0]' = [0.000e+00 ; 0.000e+00]  -> no thrust asked for
B * that           = [0.000e+00 ; 0.000e+00 ; 0.000e+00]
```

- The least-squares allocator asks for no thrust at all when it is asked for pure sway. That is the correct answer to an impossible request, and it is worth seeing before Week 7 assigns it a name.

D. The attainable control set (15 min)

```
A1_D_attainable_set
```

> **ℹ To produce this figure**
>
> `A1_D_attainable_set.m` writes `A1_simulink/img/A1_result_set.png`. The script sweeps a 121×121 grid of the shaft-speed box; it runs no simulation, because the attainable set is a property of the actuator and the geometry alone.



Reading the figure

Element	Meaning
left panel	the propeller curve, with the slope change at $n = 0$ where k switches
centre panel	the set of (X, N) reachable from the shaft-speed box
right panel	the same set viewed along the Y axis — a segment of zero width, because rank is 2

Measured over a 121×121 grid of the shaft-speed box:

Quantity	Value
$X_{\max}$	239.36 N
$X_{\min}$	-133.42 N
$\max\|N\|$	73.62 N·m
ahead / astern ratio	1.7941
convex hull area	$27\,446$ N²·m
$\max\|Y\|$ over the whole set	0, exactly

- The set is a quadrilateral with curved edges, not a box. The corners are the four combinations of saturated shaft speeds, and the edges are traced by holding one propeller at a limit and sweeping the other.

What the figure says

Three views of the same limit. The left panel is one propeller on its own, shaft speed in and thrust out. The centre panel is every generalised force the two propellers can produce between them. The right panel is that same set seen end-on, along the axis the hull cannot use.

The left curve is flat near the origin and steepens away from it because thrust is quadratic in shaft speed: doubling from 50 to 100 rad/s multiplies thrust by four, 27.70 to 110.80 N. It is also **not symmetric**. At the two saturation limits one propeller gives $+119.68$ N ahead but only -66.71 N astern, a ratio of $k_{\text{pos}}/k_{\text{neg}} = 1.7192$. Astern is simply weaker, and the centre panel inherits that lopsidedness exactly: 239.36 N ahead against -133.42 N astern.

The centre panel is a **diamond**, and its shape is not decoration. A square goes in — every combination of the two shaft speeds between their limits — and a diamond comes out, because $\mathbf{B}$ maps those two speeds onto axes at 45° to them: X is the *sum* of the thrusts and N is y_{pont} times their *difference*. The four corners of the diamond are the four saturated corners of the square.

Now compare the centre and right panels. The centre one has area; the right one has none — it is a line. That is not a plotting choice. The middle row of $\mathbf{B}$ is exactly zero, so **the sway coordinate of every attainable point is exactly zero**, and the attainable set is a flat sheet in (X, Y, N) space rather than a solid body. Every controller written in the rest of this course must choose its demand from inside that sheet.

The diamond is fixed for this hull. It does not grow when a controller is tuned harder, and it does not depend on speed or heading. It changes only when the *actuator* changes — which is precisely what the three hull variants later in the course do, and the area printed here, $27\,446$ N²·m, is the number they are measured against.

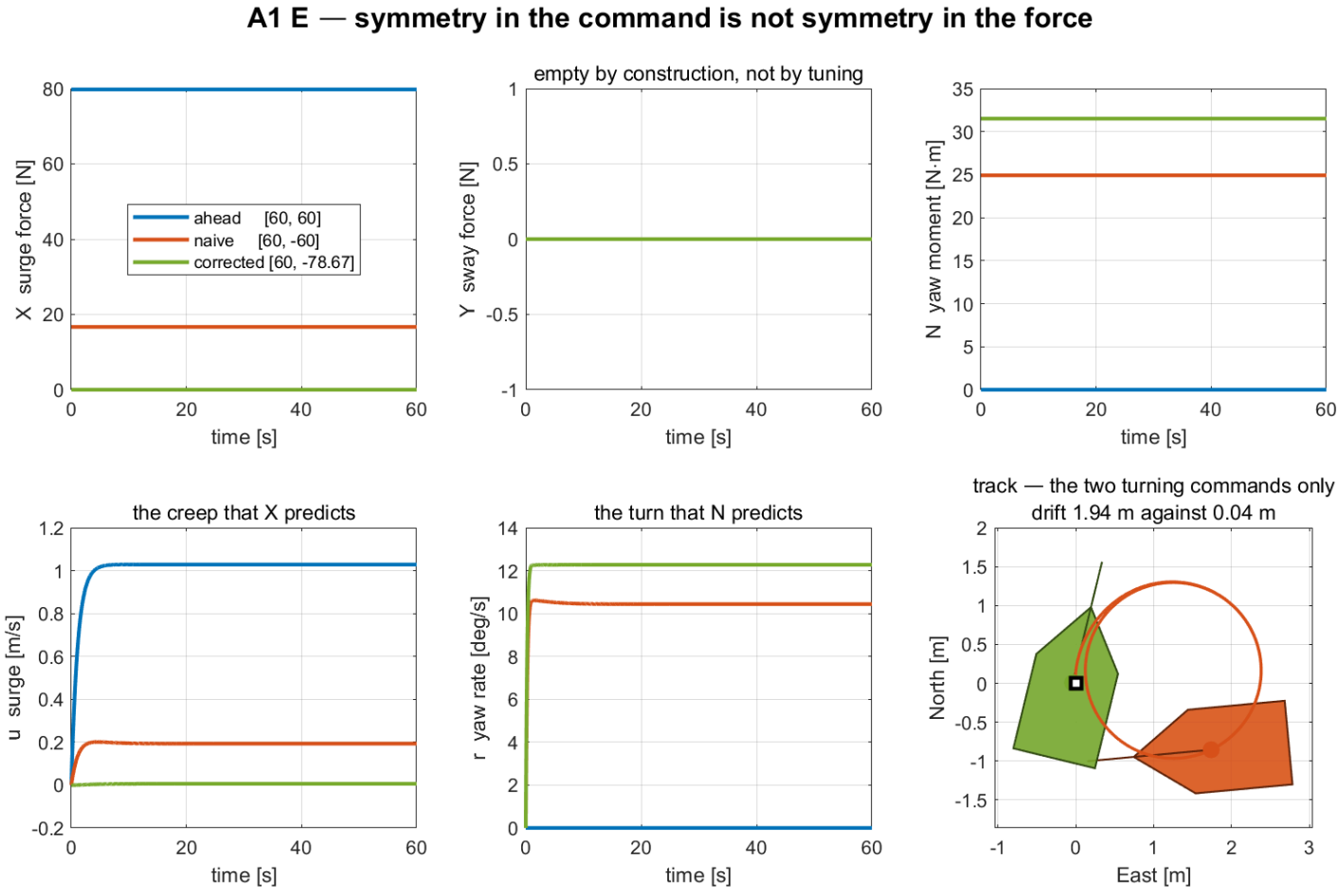
E. The command that actually turns (20 min)

```
A1_E_command_that_turns
```

> **To produce this figure**
>
> `A1_E_command_that_turns.m` runs `A1_actuation.slx` three times through `run_sim`, prints both tables below, and writes `A1_simulink/img/A1_result_turn.png`.

Three commands are applied to the same hull for 60 s each.

Command	X [N]	N [N·m]	u [m/s]	v [m/s]	r [deg/s]
ahead, $[60,\ 60]$	79.776	0.000	1.0286	0.0000	0.0000
naive, $[60,\ -60]$	16.686	24.921	0.1932	-0.0713	10.4369
corrected, $[60,\ -78.67]$	-7.1×10^{-15}	31.512	0.0062	-0.0024	12.2726



Reading the figure

Element	Meaning
top left	the surge force each command produces; the corrected one sits on zero
top centre	the sway force, plotted on a ± 1 N axis so that machine dust is not mistaken for a signal
top right	the yaw moment; the corrected command produces 26.4% more than the naive one
bottom left	the surge velocity that X predicts — the creep the correction removes
bottom centre	the yaw rate that N predicts
bottom right	the two turning tracks, each with one hull silhouette at its final pose and a filled dot at its final position; the open square is the start point

- The bottom-right panel is the result stated as a distance. After 60 s:

Command	drift from the start point	steady r
naive, $[60,\ -60]$	1.9376 m	10.4369 °/s
corrected, $[60,\ -78.67]$	0.0435 m	12.2726 °/s

- A factor of 44.6 in position error and 18% more yaw rate, from changing one number in the command.

What the figure says

The top row is what each command asks the water for; the bottom row is what the vessel does about it. Three commands, one unchanged hull — so every difference between the curves was caused by the command and nothing else.

The obvious way to turn on the spot is to run one propeller ahead and the other astern at the same speed. The orange command does exactly that, $[60, -60]$, and it does turn — 24.921 N·m of moment. But it also produces **16.686 N of surge force that nobody asked for**, and the vessel creeps forward at 0.1932 m/s while turning.

The reason is the asymmetry of §A1-5. Surge force is

$$X = k_{\text{pos}}\, n_1^2 + k_{\text{neg}}\, n_2 |n_2|,$$

and setting $n_2 = -n_1$ cancels the two **shaft speeds**, not the two **thrusts**. A propeller is not equally effective in reverse. What has to cancel is $k_{\text{pos}} n_1^2 = k_{\text{neg}} n_2^2$, which gives

$$n_2 = -n_1 \sqrt{k_{\text{pos}} / k_{\text{neg}}} = -78.6701 \ \text{rad/s}.$$

The green command uses that value, and its surge force comes out -7×10^{-15} N — zero to machine precision.

Here is the part worth pausing on. The corrected command runs its reverse propeller 31% faster, so its reverse *thrust* is larger, and the *difference* of the two thrusts — which is the moment — grows by 26.4%, to 31.512 N·m. **Removing the unwanted force made the wanted moment bigger, not smaller.** Getting the arithmetic right cost nothing and gained a quarter more turning authority.

The bottom-right panel turns all of this into a distance. The orange vessel spirals away and finishes 1.9376 m from where it began; the green one turns under itself and finishes 0.0435 m away, still sitting on the start marker. The green track is smaller than the hull that draws it, which is what "turning on the spot" means measured in metres.

The middle panel of the top row stays on zero for all three commands. No shaft speed produces sway — section D's flat diamond, seen one command at a time.

> **Why the corrected creep is not exactly zero either**
>
> The green vessel still settles at $u = 0.0062$ m/s. See the callout below: a rotating vessel has Coriolis terms that a stationary one does not, and those terms do not vanish when X does.

> **Why the corrected creep is not exactly zero**
>
> $X = 0$ holds exactly, yet u settles at 6.2 mm/s rather than at zero. The surge equation of a **rotating** vessel is not $M_{11}\dot{u} = X + X_u u$; it also contains the Coriolis term $m\,v\,r$ and a cross-flow contribution, and neither vanishes when $r \neq 0$. The residue is the size of those terms, not an error in the derivation.

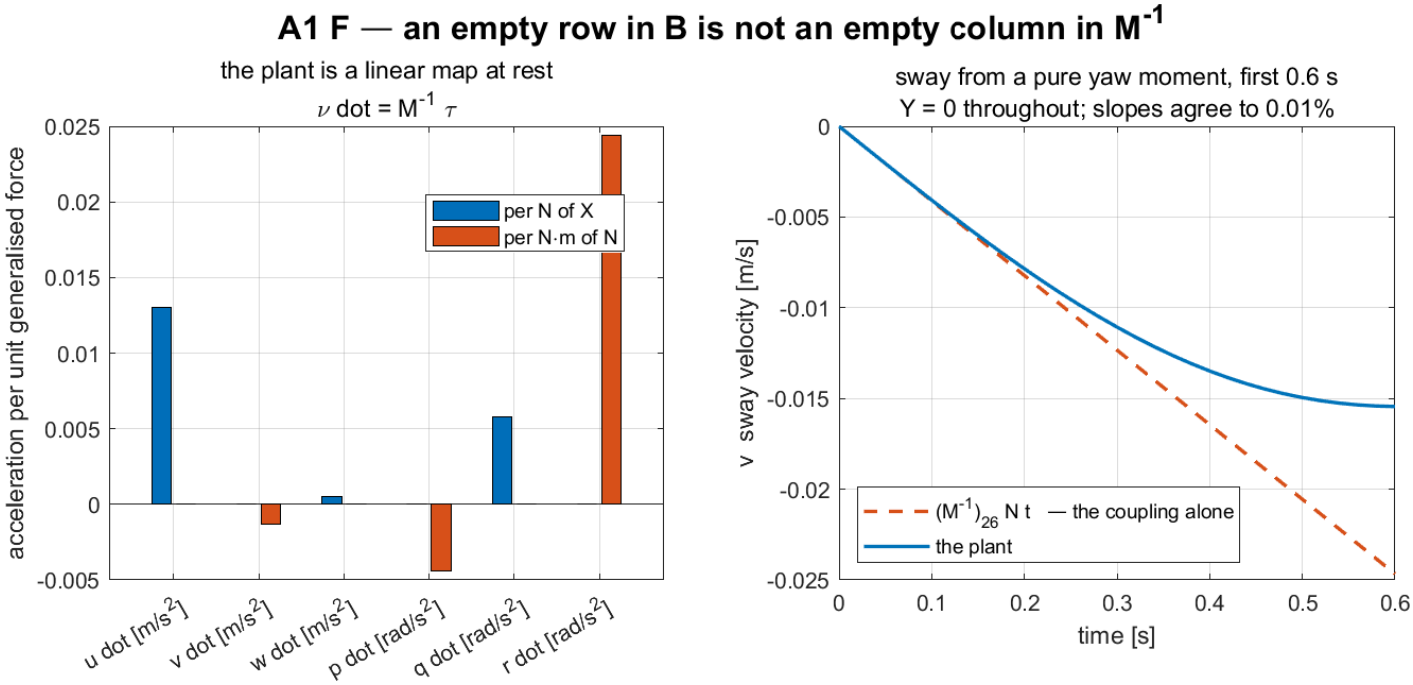
F. Sway without a sway force (15 min)

```
A1_F_sway_without_force
```

> **To produce this figure**
>
> `A1_F_sway_without_force.m` calls `otter.m` directly at $\boldsymbol{\nu} = \mathbf{0}$ for the table, runs `A1_actuation.slx` once for the right-hand panel, and writes `A1_simulink/img/A1_result_coupling.png` .

The table probes the plant at rest, where $\mathbf{C}(\boldsymbol{\nu})$ and the damping both vanish and the response is exactly $\mathbf{M}^{-1}\boldsymbol{\tau}$.

Acceleration	per N of X	per N·m of N
$\dot{u}$ [m/s²]	1.3036×10^{-2}	-2.94×10^{-18}
$\dot{v}$ [m/s²]	0	-1.3046×10^{-3}
$\dot{w}$ [m/s²]	5.2654×10^{-4}	-1.19×10^{-19}
$\dot{p}$ [rad/s²]	0	-4.3946×10^{-3}
$\dot{q}$ [rad/s²]	5.8027×10^{-3}	-1.31×10^{-18}
$\dot{r}$ [rad/s²]	0	2.4391×10^{-2}



Reading the figure

Element	Meaning
left panel	the two measured columns of $\mathbf{M}^{-1}$, one per unit of X and one per unit of N
right panel, dashed	the coupling alone, $(\mathbf{M}^{-1})_{26} Nt$
right panel, solid	the plant, over the first 0.6 s

What the figure says

The sway row of $\mathbf{B}$ is empty — and the vessel sways anyway. A pure yaw moment produces -1.3046×10^{-3} m/s² of sway per newton-metre, with no sway force anywhere in the problem.

The sway arrives through the **mass matrix** rather than through any force. **An empty row in $\mathbf{B}$ and an empty column in $\mathbf{M}^{-1}$ are different statements, and only the first is true here.**

The two blocks of zeros in the table are port–starboard symmetry: the Otter is a symmetric catamaran, so longitudinal motions and lateral motions do not mix. Inside the lateral group they do mix, through $M_{2,6} = 12.25$ kg·m, which is non-zero only because the payload sits 0.153 m forward of the origin of $\{b\}$. The entries reading 10^{-18} are arithmetic noise, not physics.

In the right panel the two curves leave the origin together, agreeing to 0.01%, because at $t = 0$ the vessel is at rest and nothing acts but $\mathbf{M}^{-1}\boldsymbol{\tau}$. They separate after about 0.15 s, once r has grown enough for the Coriolis and cross-flow terms to start opposing the sway. The dashed line keeps climbing because it is arithmetic with no physics in it after that first instant.

Read the size honestly: sway reaches -0.015 m/s and then turns back. This coupling gives a **transient** sideways motion at the start of a turn, not a way of translating sideways. Anything needing sustained sway still needs a thruster that can produce it.

> **Two effective inertias, used in later weeks**
>
> The same probe gives $1/(\mathbf{M}^{-1})_{11} = 76.71$ kg against the matrix entry $M_{11} = 85.50$ kg, and $1/(\mathbf{M}^{-1})_{66} = 41.00$ kg·m² against $M_{66} = 42.65$ kg·m². The pairs differ by 10% and 4% because the probe leaves heave, pitch and sway free while the matrix entry does not. Weeks 3 and 4 use the matrix entries and state the discrepancy rather than hiding it.

Summary

Summary

Step	What was done	How it was verified
1	Derived the column rule and rebuilt $\mathbf{B}$ from geometry	reproduces the matrix Week 1 asserted, and rank 2
2	Applied the rule to four layouts	ranks 2, 3, 3, 3 with 2, 4, 3, 8 columns, printed by `otter_config`
3	Introduced the extended thrust vector	a tilting thruster gives two columns, and `aft_azimuth` reaches rank 3 with two machines
4	Computed the attainable control set	$X \in [-133.42,\ 239.36]$ N, $\max\|N\| = 73.62$ N·m, $\max\|Y\| = 0$ exactly
5	Derived the zero-surge turning pair	$n_2 = -78.6701$ rad/s gives $X = -7 \times 10^{-15}$ N and 26.4% more yaw moment
6	Measured the drift of both turning commands	1.9376 m against 0.0435 m over 60 s
7	Probed the mass coupling at rest	$\dot{v} = -1.3046 \times 10^{-3}$ m/s² per N·m under a pure yaw moment

Progress Check

Theory

- [ ] Able to write the column of $\mathbf{B}$ for a thruster at a stated position and direction, from memory
- [ ] Able to state the rank of a layout and name the unreachable degree of freedom
- [ ] Able to explain why a tilting thruster gives two columns and what that costs
- [ ] Able to distinguish the zero row of $\mathbf{B}$ from the non-zero $\left(\mathbf{M}^{-1}\right)_{26}$

Laboratory

- [ ] `A1_0_setup` printed `rank(B) = 2` and `max |B(2,:)| = 0.0e+00`
- [ ] Sections C, D, E and F were run in that order and together wrote four PNG files into `A1_simulink/img/`
- [ ] The corrected command was run in Simulink and the live view watched to the end

Recorded observations

- [ ] X for the naive turning command recorded, and compared with zero
- [ ] The drift of both turning commands recorded at 60 s
- [ ] The initial slope of v under a pure yaw moment recorded and compared with $\left(\mathbf{M}^{-1}\right)_{26} N$

Assignment 2

- **Due**: at any point in the course
- **Submit**: the modified `A1_0_setup.m`, a derivation, the numbers requested below, and one figure

① Requirements

1. A **stern-drive** variant is proposed: the two propellers are moved aft to $x = -1.0$ m, keeping $y = \mp 0.395$ m and $\mathbf{e} = (1, 0)$. Derive $\mathbf{B}$ from the column rule and state its rank.
2. A **toe-in** variant is proposed instead: the propellers stay at $x = 0$, $y = \mp 0.395$ m, but each is canted inward by 15°, so $\mathbf{e}_1 = (\cos 15^\circ,\ +\sin 15^\circ)$ and $\mathbf{e}_2 = (\cos 15^\circ,\ -\sin 15^\circ)$. Derive $\mathbf{B}$ and state its rank.
3. For the toe-in variant, determine by hand whether the sway row is still empty, and if not, what the largest attainable $|Y|$ is.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the averaging window stated**

1. Implement both variants by editing `B_alloc` in `A1_0_setup.m`, and report `rank(B_alloc)` for each as printed by MATLAB.
2. For the toe-in variant, compute and report the attainable control set: $X_{\max}$, $X_{\min}$, $\max|Y|$, $\max|N|$. Produce **one figure** showing the set projected on (X, N) and on (X, Y).
3. Apply the naive command $[60,\ -60]$ to the stern-drive variant and report the settled u, v and r. State whether the yaw moment changed relative to the base layout, and by how much.
4. For the toe-in variant, report the loss in $X_{\max}$ relative to the base layout, in newtons and as a percentage.

③ Analysis (5–10 lines)

- Item ①.2 produces a sway row that is no longer empty, and item ②.4 produces a loss of forward thrust. State whether the toe-in variant is fully actuated in the plane, argue the answer from the rank rather than from the magnitudes, and say what the loss of forward thrust buys. Compare the result with `bow_thruster`, which reaches rank 3 by a different route.

Grading

Criterion	Weight
Both derivations in ① are complete and correct	20%
Verification performed and numbers reported with windows stated	40%
The figure in ②.2 is correct and readable	15%
Analysis in ③ argues from rank, not from magnitude	25%

Troubleshooting

Symptom	Cause	Fix
`Gain` block error: *Matrix dimensions must agree*	`B_alloc` was edited to a size that does not match the number of thrusters	$\mathbf{B}$ must be $3 \times n$ and the thruster block must output n values
The sway row is 10^{-17} rather than 0	a direction was entered as an angle and evaluated with `cosd` / `sind`	it is still exactly zero in `otter_B`, which uses exact $(1, 0)$; the residue comes from the hand entry
$\mathbf{B}$ has the opposite sign in the yaw row	the moment was taken as $y\,e_x - x\,e_y$	the column rule is $x\,e_y - y\,e_x$, the z component of $\mathbf{r} \times \mathbf{e}$
The corrected command still creeps forward	expected	$X = 0$ exactly, but the surge equation of a rotating vessel also contains Coriolis and cross-flow terms
The two turning tracks look identical in the live view	the axis window is set for a 30 m box	the tracks differ by 2 m; narrow `track_Nmin` … `track_Emax` in `A1_0_setup.m`
`rank(B)` reports 3 for the base layout	`B_alloc` was replaced by a variant and not restored	run `A1_0_setup` again

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. §11.2 (control allocation) and §12.3.4 (the extended thrust vector).
- MSS toolbox, `Tools/MSS/VESSELS/otter.m` — the propeller model and lever arms, lines 91–98 and 181.
- Fossen, T. I. and Johansen, T. A. "A survey of control allocation methods for ships and underwater vehicles." *14th Mediterranean Conference on Control and Automation*, 2006.

Course files

- `A1_simulink/A1_1_build_actuation.m` — the model generator
- `A1_simulink/A1_0_setup.m` — the parameters, including `B_alloc`
- `A1_simulink/A1_C_four_layouts.m` · `A1_D_attainable_set.m` · `A1_E_command_that_turns.m` · `A1_F_sway_without_force.m` — one script per laboratory section
- `A1_simulink/A1_vars.m` · `A1_read.m` — the same numbers as a struct, and the log with named fields
- `A1_simulink/A1_animate.m` — the live view
- `_tools/otter_B.m` — the column rule, the single definition used by this course
- `_tools/otter_config.m` — the four layouts

Where this is used

Result of this appendix	Used by
$X \in [-133.42,\ 239.36]$ N	Week 3, as the saturation that causes integrator windup
$N = y_{\text{pont}}(T_1 - T_2)$	Week 4, as the yaw allocation
$n_2 = -n_1\sqrt{k_{\text{pos}}/k_{\text{neg}}}$	Week 4, which produces the same pair automatically by allocating **thrust** rather than shaft speed
the column rule and $\mathbf{B}^{\dagger}$	Week 7, control allocation
the four layouts and their ranks	Weeks 11 to 12, the hull variants